

Documentación de Arquitectura del Sistema C5 Universitario de Alerta Ciudadana

1. Introducción

El presente documento describe la arquitectura propuesta para un **Sistema de Alerta Ciudadana tipo C5 (Centro de Comando, Control, Comunicación y Cómputo)** aplicado a un entorno universitario.

El sistema tiene como objetivo recibir alertas generadas por dispositivos IoT distribuidos dentro del campus, procesarlas mediante una arquitectura basada en microservicios, determinar la prioridad de cada evento, almacenar el historial de incidentes y notificar en tiempo real a los operadores encargados de la atención.

La arquitectura utiliza tecnologías de comunicación IoT, procesamiento distribuido, almacenamiento persistente y comunicación en tiempo real para lograr una solución escalable y tolerante a fallos.

2. Objetivo del Sistema

Diseñar e implementar una plataforma capaz de:

- Recibir alertas desde botones de emergencia físicos ESP32.
- Procesar eventos en tiempo real.
- Clasificar la prioridad de una emergencia.
- Obtener la ubicación del dispositivo que genera la alerta.
- Registrar los eventos ocurridos.
- Notificar inmediatamente a operadores mediante un dashboard.
- Mantener disponibilidad del sistema mediante servicios independientes.

3. Arquitectura General del Sistema

La arquitectura está basada en un modelo distribuido utilizando:

- Dispositivos IoT
- Comunicación mediante MQTT
- Microservicios independientes
- Cola de mensajes
- Base de datos replicada
- Comunicación WebSocket en tiempo real

El flujo general del sistema es:

ESP32 → MQTT Broker → Gateway NGINX → Microservicios → Redis → Servicios internos → Dashboard

4. Componentes de la Arquitectura

4.1 Dispositivos IoT (ESP32 Tótem)

Los dispositivos ESP32 representan los botones físicos de emergencia instalados dentro de la universidad.

Su función es:

- Detectar cuando un usuario activa una alerta.
- Capturar información del evento.
- Enviar mensajes mediante protocolo MQTT.

La información enviada puede contener:

- ID del dispositivo.
- Tipo de alerta.
- Fecha y hora.
- Ubicación.
- Estado del botón.

Ejemplo:

```
{  
  "dispositivo":"ESP32-01",  
  "evento":"PANICO",  
  "ubicacion":"Edificio A",  
  "hora":"10:30"  
}
```

4.2 Protocolo MQTT

MQTT (Message Queuing Telemetry Transport) es el protocolo utilizado para comunicación entre dispositivos IoT y el servidor.

Se eligió porque:

- Tiene bajo consumo de recursos.
- Funciona correctamente con dispositivos pequeños.
- Permite comunicación rápida.
- Utiliza modelo Publisher / Subscriber.

Funcionamiento:

El ESP32 actúa como:

Publisher

publicando alertas.

El servidor actúa como:

Subscriber

recibiendo los eventos.

4.3 Mosquitto MQTT Broker

Mosquitto funciona como intermediario de comunicación.

Sus responsabilidades son:

- Recibir mensajes de los ESP32.
- Administrar los temas MQTT.
- Entregar mensajes a los servicios suscritos.

Ejemplo de tópico:

universidad/emergencias

Cuando un ESP32 genera una alerta:

```
ESP32
|
| MQTT
|
Mosquitto
|
Microservicios
```

4.4 NGINX Gateway

NGINX funciona como puerta de entrada del sistema.

Funciones:

- Recibir conexiones externas.
- Distribuir solicitudes.
- Balancear tráfico hacia los servicios internos.
- Servir como punto único de acceso.

Ayuda a que los microservicios estén protegidos y separados.

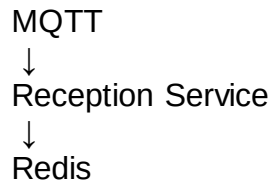
4.5 Servicio de Recepción (Reception Service)

Este servicio recibe las alertas provenientes del broker MQTT.

Funciones:

- Escuchar mensajes MQTT.
- Validar la información recibida.
- Transformar el evento.
- Enviar la alerta hacia Redis.

Comunicación:



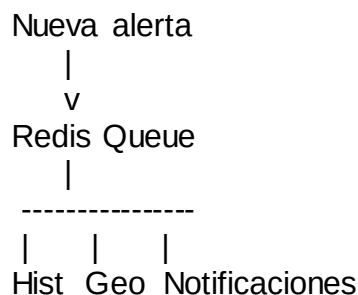
4.6 Redis como sistema de mensajería

Redis funciona como intermediario rápido entre servicios.

Se utiliza para:

- Guardar eventos temporalmente.
- Crear colas de procesamiento.
- Distribuir información entre microservicios.

Ejemplo:



4.7 History Service

Este servicio administra el historial de eventos.

Funciones:

- Registrar alertas.
- Consultar eventos anteriores.
- Generar reportes.

Comunicación:

Redis
|
History Service
|
PostgreSQL

4.8 PostgreSQL Master - Réplica

Se utiliza una arquitectura de base de datos con replicación.

Master

Realiza:

- Escrituras.
- Inserciones de nuevos eventos.

Ejemplo:

INSERT alerta

Réplica

Realiza:

- Lecturas.
- Consultas históricas.

Ejemplo:

SELECT alertas

Ventajas:

- Mayor disponibilidad.
- Menor carga sobre la base principal.
- Recuperación ante fallos.

4.9 Geolocation Service

Este servicio determina la ubicación del evento.

Funciones:

- Identificar dónde ocurrió la alerta.
- Obtener coordenadas.
- Asociar el dispositivo con una zona.

Ejemplo:

ESP32-05

|
v

Edificio B - Segundo piso

Utiliza comunicación:

gRPC

para comunicarse con otros servicios internos.

4.10 Priority Service

Su función es determinar la gravedad del evento.

Clasifica las alertas:

Ejemplo:

Tipo	Prioridad
Botón de pánico	Alta
Incidente menor	Media
Información	Baja

Puede usar reglas:

Si evento = PANICO

Prioridad = CRITICA

4.11 Transcription Service

Servicio encargado del procesamiento de texto.

Funciones:

- Recibir mensajes.
- Convertir información.
- Publicar resultados mediante canales internos.

Comunicación:

Pub/Sub

4.12 Notifications Service

Es el servicio encargado de avisar a los operadores.

Funciones:

- Recibir alertas procesadas.
- Enviar información al dashboard.
- Mantener comunicación activa.

Utiliza:

WebSocket

para comunicación en tiempo real.

4.13 Dashboard de Operadores

Es la interfaz utilizada por el personal encargado.

Permite:

- Visualizar alertas.
- Ver ubicación.
- Revisar prioridad.
- Consultar historial.

La información aparece inmediatamente cuando ocurre una emergencia.

5. Flujo completo de una alerta

Ejemplo:

1. Un estudiante presiona un botón de emergencia.

2. El ESP32 genera una alerta.
3. La alerta viaja mediante MQTT hacia Mosquitto.
4. Mosquitto entrega el mensaje al servicio Reception.
5. Reception almacena temporalmente el evento en Redis.
6. Los servicios procesan la información:
 - Historial guarda el evento.
 - Geolocation obtiene ubicación.
 - Priority clasifica gravedad.
 - Notifications prepara aviso.
7. Dashboard recibe la alerta mediante WebSocket.
8. El operador visualiza el incidente.

6. Ventajas de la Arquitectura

Escalabilidad

Se pueden agregar más ESP32 o servicios sin modificar todo el sistema.

Tolerancia a fallos

Si un microservicio falla, los demás continúan funcionando.

Procesamiento en tiempo real

Las alertas llegan inmediatamente al operador.

Seguridad

Los servicios están separados y protegidos mediante un gateway.

Disponibilidad

La base de datos cuenta con replicación.

7. Tecnologías utilizadas

Componente	Tecnología
Dispositivo IoT	ESP32
Comunicación IoT	MQTT
Broker	Mosquitto
Gateway	NGINX
Backend	Microservicios
Mensajería	Redis
Base de datos	PostgreSQL
Comunicación interna	gRPC
Tiempo real	WebSocket
Contenedores	Docker

